GMSH

Gmsh est un logiciel libre de génération de maillages (mesh generator) développé initialement par Christophe Geuzaine et Jean-François Remacle. Il est largement utilisé dans la modélisation numérique et les méthodes de calcul par éléments finis (FEM), volumes finis (FVM) ou autres approches discrètes.

Ses principales caractéristiques :

Interface graphique et langage de script : Gmsh peut être utilisé via une interface graphique interactive, mais aussi à travers des fichiers de script (.geo) permettant de définir géométrie, maillage et conditions aux limites.

Support de géométries complexes : il permet de décrire des géométries allant des formes simples (lignes, surfaces, volumes) à des structures 3D très complexes.

Génération de maillages 1D, 2D et 3D : Gmsh produit des maillages structurés ou non structurés avec des éléments de type segment, triangle, quadrangle, tétraèdre, hexaèdre, prisme ou pyramide.

Maillage adaptatif : possibilité de raffiner localement le maillage en fonction d'un champ (par exemple un gradient de solution numérique).

Interopérabilité : nombreux formats d'exportation du maillage existent en particulier .msh .

En résumé, Gmsh est un outil polyvalent et accessible qui sert de passerelle entre la définition géométrique et la simulation numérique, très apprécié dans le monde académique et industriel grâce à sa flexibilité et à sa gratuité.

Exemple de script geo pour décrire la géométrie

Script (.geo) pour décrire la géométrie

```
cl1 = 0.05;  cl2 = 0.05; cl3 = 0.05;

L = 1;
R=0.5;

Point(1) = {0, 0, 0, cl1};
Point(2) = {+R, 0, 0, cl1};
…
Point(13) = {+0, -L, 0, cl2};

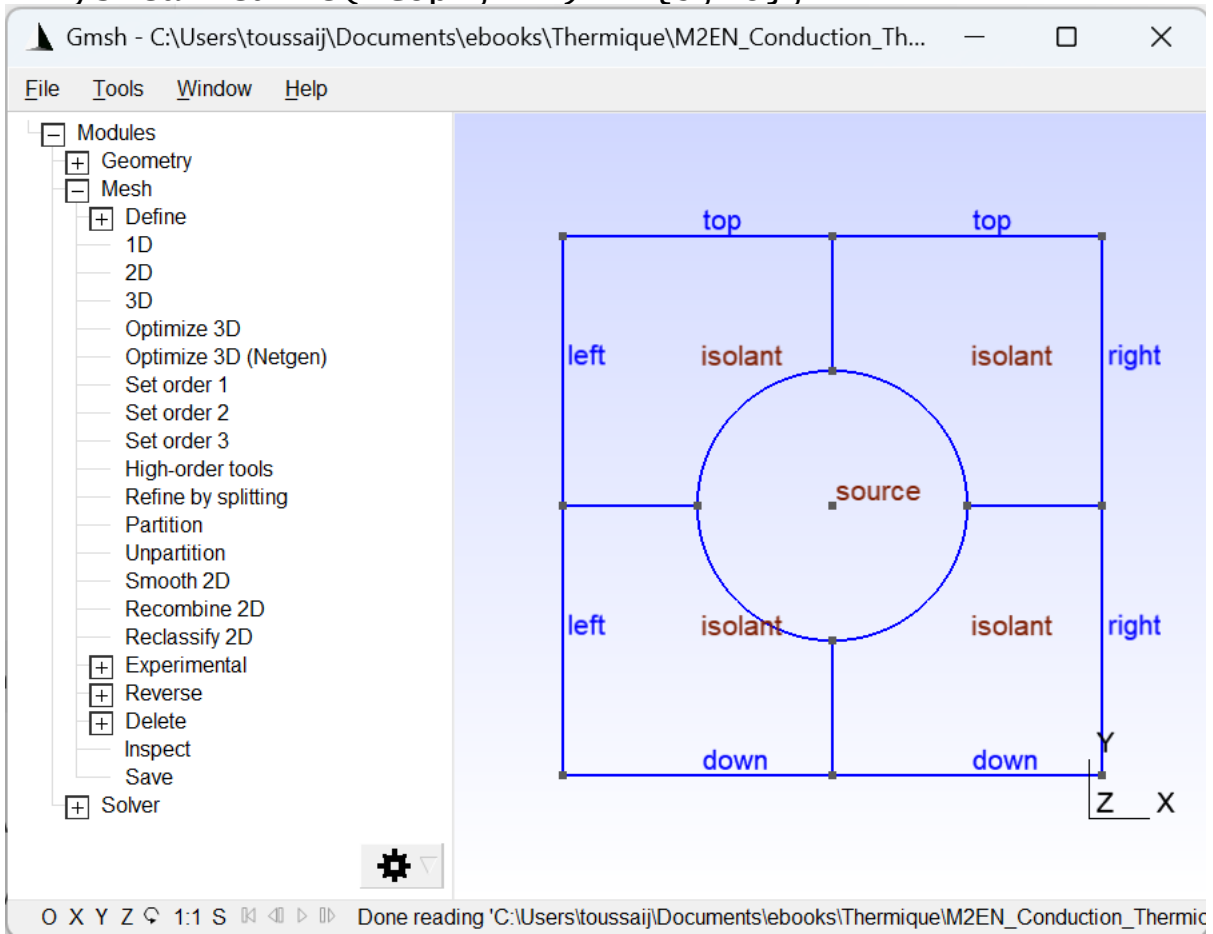
Circle(1) = {2, 1, 4};
…
Circle(4) = {5, 1, 2};

Line(5) = {13, 7};
…
Line(16) = {5, 13};

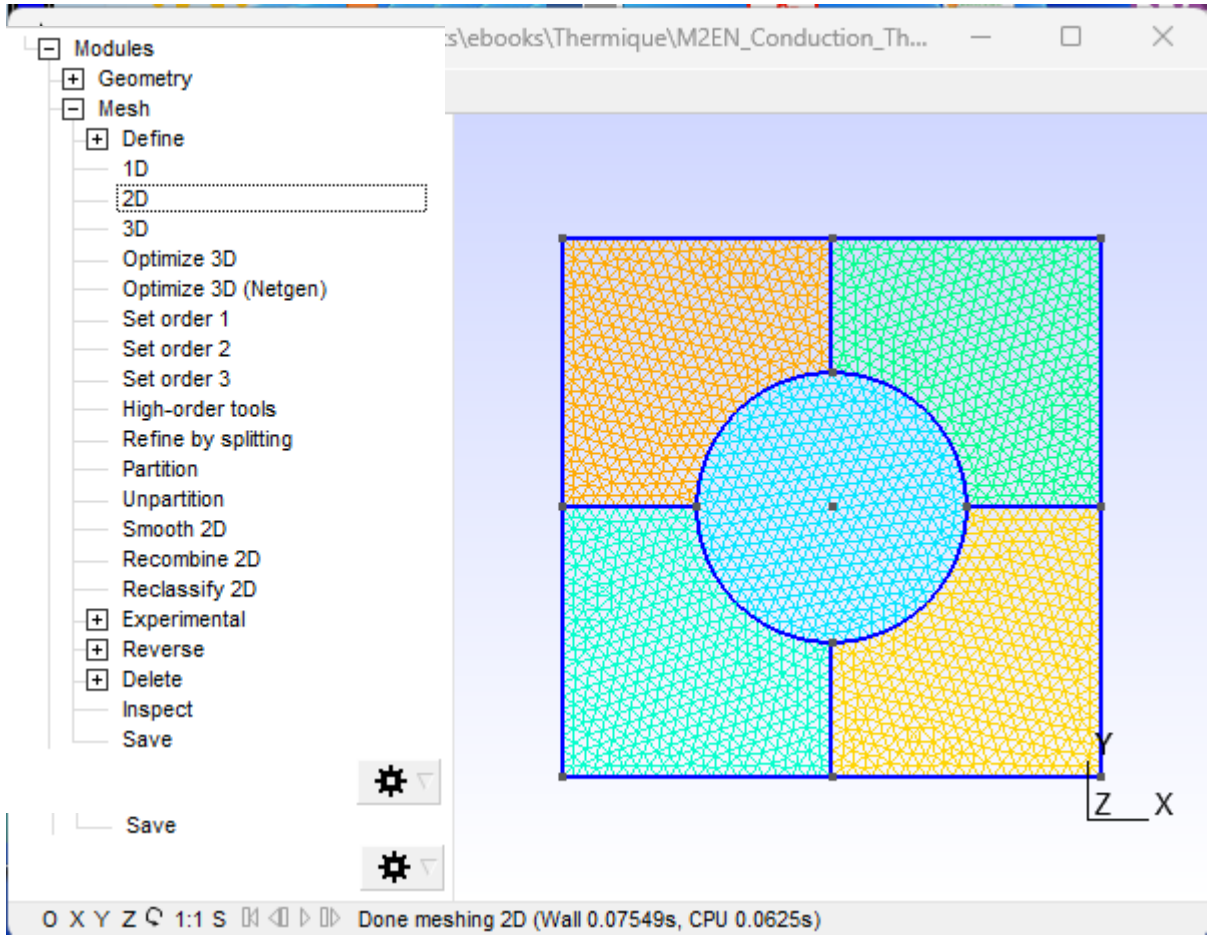
Curve Loop(1) = {13, 7, 8, -14, -1};
Plane Surface(1) = {1};
…
Curve Loop(5) = {2, 3, 4, 1};
Plane Surface(5) = {5};
```

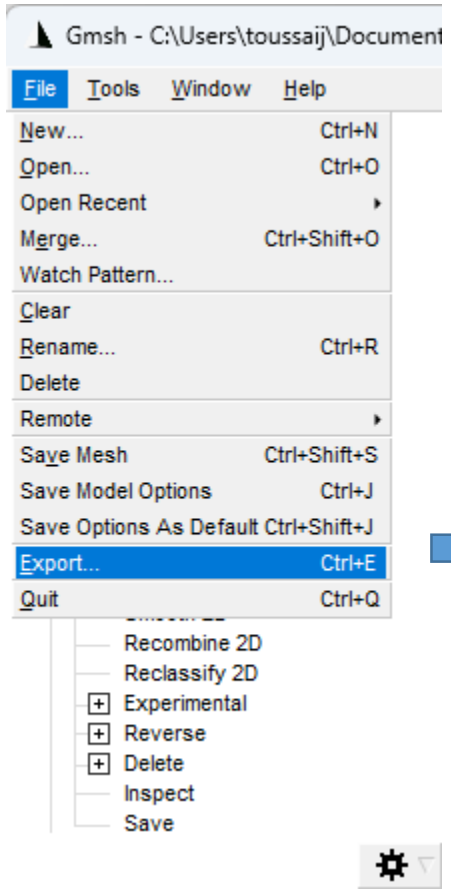
```
Physical Surface("source", 17) = {5};
Physical Surface("isolant", 22) = {2, 1, 4, 3};

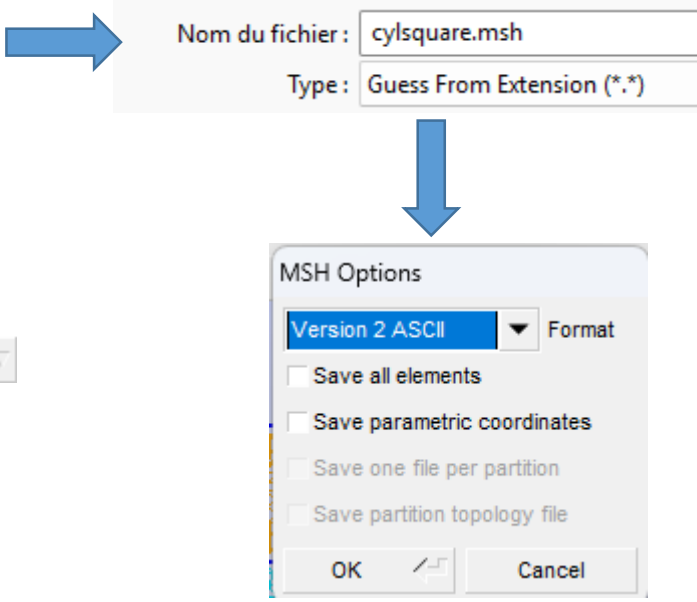
Physical Curve("left", 18) = {10, 11};
Physical Curve("down", 19) = {12, 5};
Physical Curve("right", 20) = {7, 6};
Physical Curve("top", 21) = {9, 8};
```



Maillage conforme en triangles et segments







Assign_BCs : CONDITIONS AUX LIMITES

```python
from FV_lib.struct import Struct

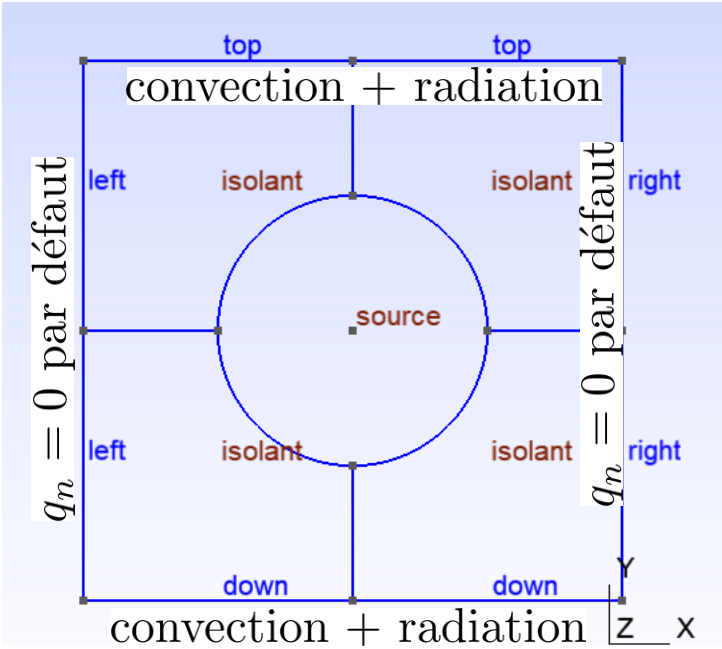
def assign_BCs(cfd):
    """
    ASSIGN_BCS - Assigne les conditions aux limites de Dirichlet et de robinn
    """
    # Dictionnaire pour les conditions de Dirichlet : clé = nom de région, valeur = scalaire
    cfd.dirichlet_map = {}

    # Exemples de conditions de Dirichlet (commentés comme dans l'original)
    # cfd.dirichlet_map['left'] = Struct(Td = lambda x, y: 300)
    # cfd.dirichlet_map['right'] = Struct(Td = 500)

    # Dictionnaire pour les conditions de Robin
    cfd.robin_map = {}
    cfd.robin_map['top']   = Struct(hcv=10, Ta=300)
    cfd.robin_map['down']  = Struct(hcv=10, Ta=300)

    # Dictionnaire pour les conditions de Stefan
    cfd.stefan_map = {}
    cfd.stefan_map['top']   = Struct(eps=0.8, Tsur=350)
    cfd.stefan_map['down']  = Struct(eps=0.8, Tsur=350)

    return cfd
```



Assign_properties : PROPRIETES PHYSIQUES

```python
from FV_lib.struct import Struct

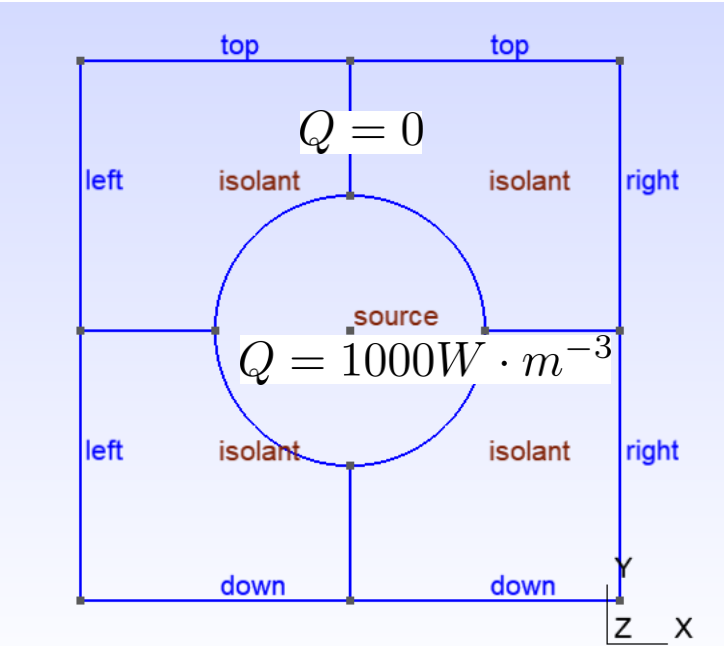
def assign_properties(cfd):
    """
    Assigne les propriétés des matériaux aux régions du maill

    Args:
        cfd: objet Struct contenant les données du maillage

    Returns:
        cfd: objet Struct avec les propriétés assignées
    """

    # Dictionnaire pour les propriétés : clé = nom de région, valeur = dict des propriétés
    cfd.properties_map = {}

    # Définition des propriétés par région
    cfd.properties_map['isolant']   = Struct(K=60,  Q = lambda x, y: 0)
    cfd.properties_map['source']    = Struct(K=240, Q = lambda x, y: 1000)
...
    return cfd
```



solve_linear_system : SOLVEUR LINEAIRE

```python
from FV_lib.struct import Struct
import numpy as np
import scipy.sparse.linalg
from scipy.sparse import csr_matrix
from FV_impl.matrix import build_diffusion_matrix, apply_BCs_to_diffusion,
       apply_periodic_BCs_to_diffusion_matrix

def solve_linear_system(cfd):
    """
    Résout le système linéaire et stocke la solution dans cfd.T
    ""«

    print(f"\nMatrice de diffusion construite:")
    print(f"  - Taille: {cfd.MD.shape}")
    print(f"  - Éléments non-nuls: {cfd.MD.nnz}")
    print(f"  - Sparsité: {cfd.MD.nnz / (cfd.MD.shape[0] * cfd.MD.shape[1]) * 100:.2f}%")

    # Résolution du système linéaire
    cfd.T = scipy.sparse.linalg.spsolve(cfd.MD, cfd.VD + cfd.VQ)

    return cfd
```

solve_nonlinear_system :

SOLVEUR NON-LINEAIRE

```
def solve_nonlinear_system(cfd, Tguess=300):
    """
    Résout le système non-linéaire et stocke la solution dans cfd.T
    """
    cfd.T = Tguess*np.ones(len(cfd.cell))
    Told = cfd.T.copy()
    niter, rtol = 0, 1e-8
    while True:
        niter +=1
        # Construit la matrice de diffusion
        cfd = build_diffusion_matrix(cfd)

        # Applique les conditions aux limites Dirichlet ou Robin
        cfd = apply_BCs_to_diffusion(cfd)

        # Résolution du système linéaire
        cfd.T = scipy.sparse.linalg.spsolve(cfd.MD, cfd.VD + cfd.VQ)

        print(f'Temperature min : {np.min(cfd.T)} max : {np.max(cfd.T)}')
        err = np.linalg.norm(cfd.T-Told, ord=np.inf)/np.linalg.norm(Told, ord=np.inf)
        if abs(err) < rtol: break
        Told = cfd.T.copy()

    print(f"nombre d'itérations : {niter}")
    return cfd
```

py_FV_main : principal (1/2)

```
if __name__ == "__main__":
    cfd = Struct()
    try:
        cfd.mesh_name = select_file()
        cfd = read_gmsh_msh(cfd)

        # Affiche le maillage et les régions surfaciques
        plot_regions(cfd)

        # Affiche les cellules
        plot_cells(cfd)

        # Définit les propriétés physiques par région
        cfd = assign_properties(cfd)

        # Définit les conditions aux limites
        cfd = assign_BCs(cfd)

        # Construit le vecteur source
        cfd = build_source_vector(cfd)
```

py_FV_main : principal (2/2)

```
    # Résout le système linéaire
    cfd = solve_nonlinear_system(cfd)

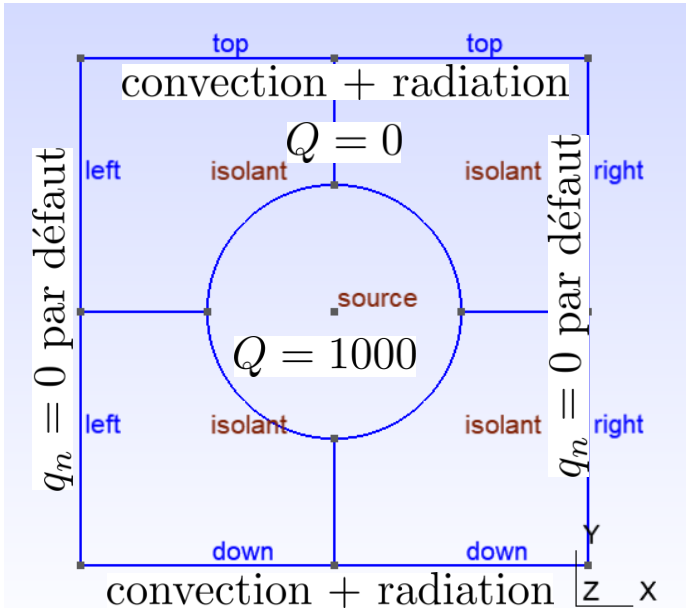
    # Représente les données brutes
    plot_raw_sol(cfd)

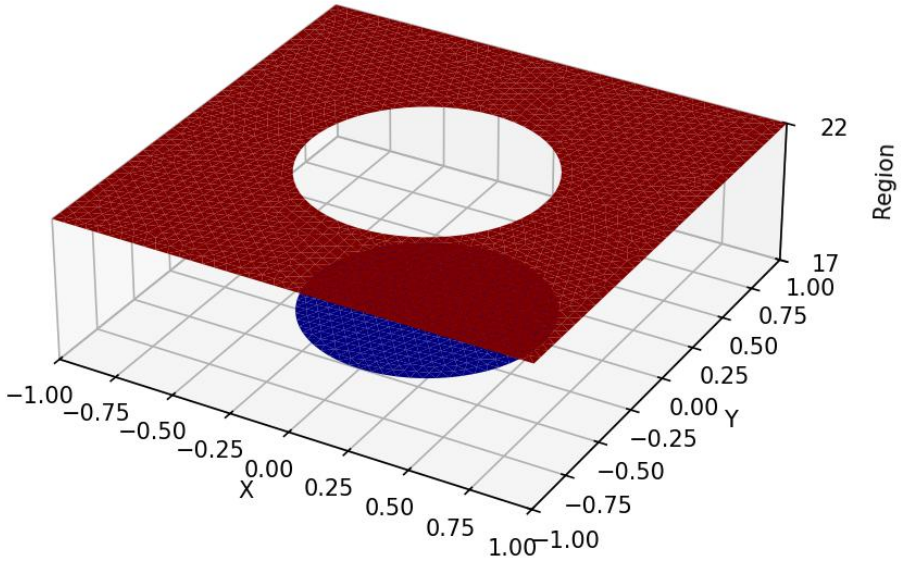
    # Représente les données lissés et isovaleurs
    plot_smoothed_sol(cfd)

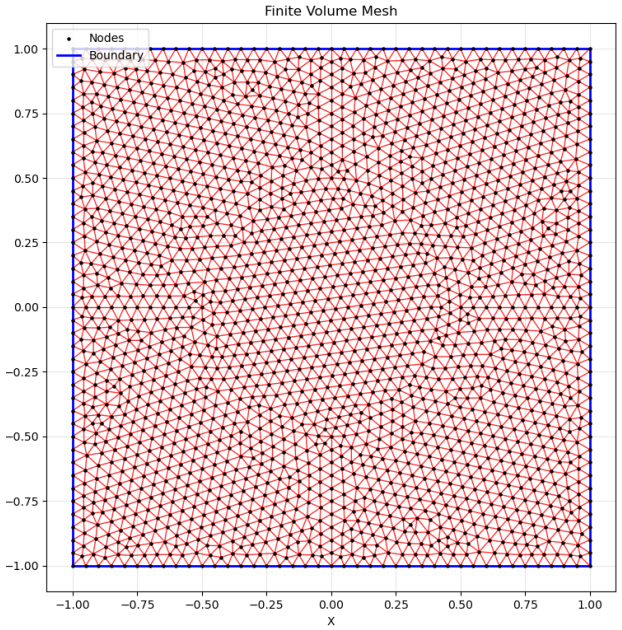
    # Trace le profil de température de [xmin, 0] à [xmax, 0]
    cutline(cfd, 100, 0, cfd.ymin, 0, cfd.ymax)

except Exception as e:
    print(f"Erreur lors de la lecture du maillage: {e}")
    import traceback
    traceback.print_exc()
```

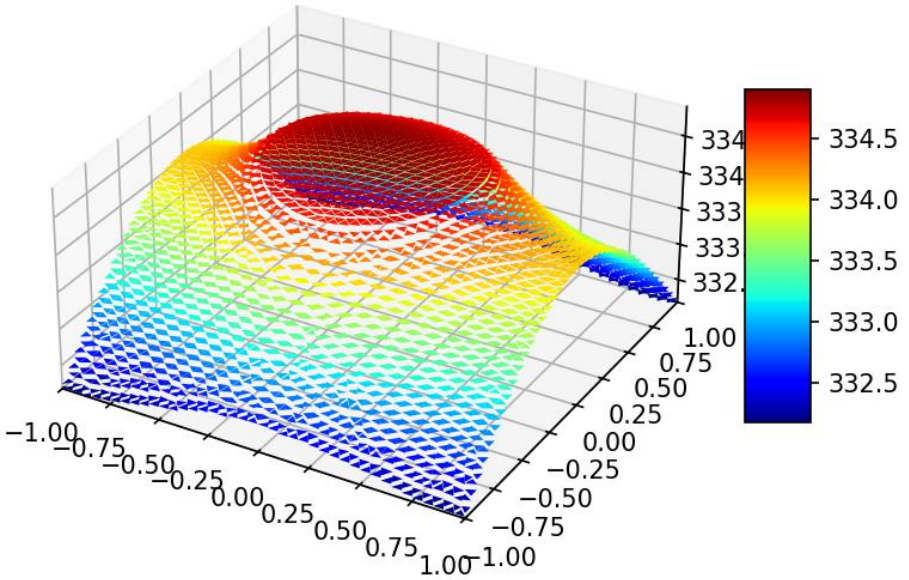
Simulation de la température



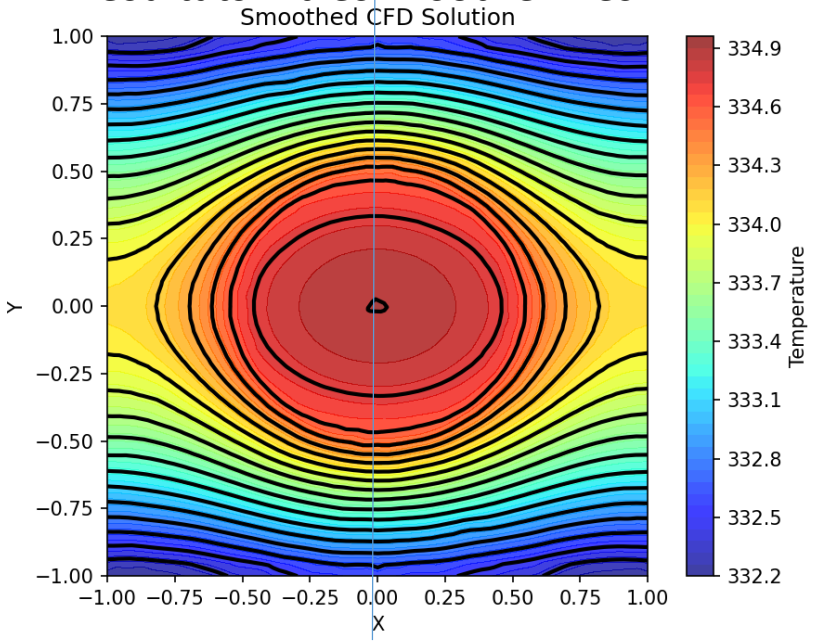




Résultats bruts



Résultats filtrés + isothermes



Evolution selon Oy

